

基于昇腾边缘计算平台的锂电池寿命预测系统-高频问题

一、口语化项目介绍

1. 常规项目介绍版

面试官：你介绍一下这个锂电池寿命预测项目。

这个项目主要面向BMS中的电池健康管理和剩余寿命预测，包含算法研究和昇腾端侧部署两条主线。

锂电池随着充放电循环会逐渐老化，表现为容量下降和性能衰减。SOC只能告诉我们电池当前还剩多少电，但对于维护和更换决策来说，还需要知道电池已经老化到什么程度，以及距离寿命终点还有多少个循环，也就是SOH、EOL和RUL。

这个问题的难点在于，锂电池退化是一个长期、非线性的过程。原始充放电数据中包含电压、电流、温度和时间等多维信息，但不同特征对容量衰减的敏感程度不同。如果把所有特征直接输入模型，可能会引入弱相关信息和特征冗余。

因此，在算法侧，我首先基于NASA公开电池数据，从充电、放电和温度变化中提取了11维退化特征；然后使用GRA计算各特征与容量退化趋势的关联程度，以0.75为阈值保留9维强相关特征；再通过PCA将其压缩成4维健康因子，前四个主成分累计方差贡献率达到98.65%。

之后，我把连续多个循环的健康因子输入LSTM，利用LSTM学习电池长期、非线性的容量退化规律。模型直接输出的是未来容量值，再根据容量预测曲线越过EOL阈值的位置计算EOL和RUL。实验中还比较了不同健康因子构造方式以及21步、42步两种预测策略。

在完成PC端算法以后，我又把模型部署到了华为Atlas 200I DK A2。这里主要解决了PyTorch动态图和昇腾静态编译之间的差异。我完成了 `.pth` $\rightarrow$ `ONNX` $\rightarrow$ `OM` 的模型转换，将输入固定为 `[1, 21, 4]`，并把LSTM的 `h0/c0` 显式设置为输入，把 `hn/cn` 设置为输出，由板端程序在多次推理之间维护状态。

在板端，我基于ACL Lite搭建了从资源初始化、OM加载、数据标准化、模型执行、容量反标准化，到指标计算和资源释放的完整推理链路，同时完成了ARM64依赖、无图形界面结果保存和命令行多模型切换等适配。

最终，后期对五组电池进行了全周期容量预测，平均RMSE不超过0.004 Ah，项目自定义准确率超过99.8%，EOL判断误差小于2个循环。相关算法研究以第一作者发表了EI会议论文。

不过，这个项目目前仍属于公开数据集和边缘部署验证阶段，还没有接入真实量产BMS，所以我不会把它描述成车规级部署成果。

2. 30秒概括版

这个项目主要做锂电池容量和剩余寿命预测，并把PC端算法部署到昇腾边缘设备。

算法侧，我从NASA电池数据中提取了11维退化特征，通过GRA筛选出9维强相关特征，再利用PCA压缩成4维健康因子，最后使用LSTM预测容量曲线，并进一步判断EOL和RUL。

部署侧，我完成了 `.pth` $\rightarrow$ `ONNX` $\rightarrow$ `OM` 转换，通过固定Shape和显式 `h0/c0` 解决LSTM静态图部署问题，并基于ACL Lite在Atlas 200I DK A2上实现完整板端推理。后期五组电池平均RMSE不超过0.004 Ah，EOL误差小于2个循环，相关算法以第一作者发表EI会议论文。

二、高频技术追问题库

1. 你在这个项目中主要负责什么？

我主要负责算法研究和昇腾端侧部署。

算法侧包括NASA电池数据处理、11维退化特征提取、GRA筛选、PCA降维、LSTM模型构建、容量预测和EOL/RUL分析。

部署侧包括PyTorch模型导出ONNX、ATC编译OM、固定Shape适配、LSTM状态接口改造，以及基于ACL Lite完成板端推理工程。

论文写作和研究讨论有课题组老师与合作者参与，我主要负责算法实现、实验分析和后续昇腾部署。

2. 为什么要把模型部署到边缘端？云端推理不行吗？

云端推理当然可以，而且在模型集中管理和训练方面更有优势。

这个项目做边缘部署，主要不是追求毫秒级响应，因为RUL变化速度没有那么快。核心价值是验证模型能否在靠近BMS的设备上独立运行，减少对网络连接的依赖，同时降低原始电池数据长期上传带来的通信和隐私压力。

更实际的产品形态可能是边云协同：边缘端负责本地预测，云端负责模型训练、版本管理和长期数据分析。

3. 为什么不直接把原始电压、电流和温度曲线输入模型？

NASA数据规模相对有限，而且不同循环的原始采样序列长度不一致。如果直接输入完整曲线，需要进行重采样或填充，模型规模和过拟合风险也会增加。

所以我先提取具有一定物理意义的充放电退化特征，把每个循环转换为固定维度向量。这样既能减少输入规模，也更方便解释和部署。

代价是人工特征可能损失原始曲线中的局部信息。如果数据规模更大，可以进一步比较1D CNN、TCN或Transformer等端到端方法。

4. 为什么GRA后还要做PCA？

GRA和PCA解决的是两个不同问题。

GRA使用容量作为参考序列，判断哪些特征与容量退化趋势更相关，主要用于剔除弱相关特征。

PCA不使用容量标签，而是分析保留特征之间的方差和共线性，把相互关联的特征压缩为少数正交主成分。

所以我的思路是先通过GRA去掉弱相关特征，再通过PCA减少强相关特征之间的冗余。最终从11维筛选到9维，再压缩成4维健康因子。

5. 为什么GRA阈值选择0.75？

0.75是项目中用于筛选强相关特征的经验阈值。关联度高于该值，说明特征变化趋势与容量退化趋势比较接近。

它并不是一个对所有数据都最优的固定值。更严格的做法是对不同阈值进行敏感性分析，并结合验证集RMSE和EOL误差选择阈值。

所以我会把0.75描述为项目实验设定，而不会说它具有普适性。

6. 为什么使用LSTM，而不是普通RNN？

电池退化是一个长期时序过程，当前容量状态与之前多个循环的退化历史有关。

普通RNN在长序列反向传播过程中容易出现梯度消失或梯度爆炸。LSTM通过细胞状态和输入门、遗忘门、输出门控制信息传递，能够缓解长期依赖学习困难。

这里应该说“缓解梯度问题”，而不是说LSTM完全解决了梯度消失和梯度爆炸。

7. 模型是直接预测RUL吗？

不是。模型直接预测的是未来容量值。

我首先根据健康因子序列得到容量预测曲线，再找到预测容量第一次越过EOL阈值的循环位置，得到预测EOL。预测EOL与当前循环之差就是RUL。

所以这个项目本质上是容量回归，然后由容量曲线推导EOL和RUL。

8. 21步预测和21步输入窗口是一个概念吗？

不是。

21步预测表示预测跨度，也就是向未来预测21个循环的容量；21步输入窗口表示每次模型推理使用最近21个循环的健康因子。

两者在部署版本中都使用了21这个数值，但含义不同。一个描述预测目标距离，一个描述模型能够看到的历史范围。

9. 为什么回归任务会有“准确率”？

这里的准确率不是分类任务中的正确率，而是项目自定义的相对误差指标。

它使用MAE除以真实容量均值，再用1减去这个比例。这个指标表示平均绝对误差相对于平均容量的大小。

因此，我在面试中会主动给出计算方式，同时配合RMSE、MAE、 R^2 和EOL循环误差，不会只使用“准确率”描述回归结果。

10. 如何避免时间序列数据泄漏？

首先应该按照电池编号或者时间顺序划分训练集和测试集，而不是在生成高度重叠的窗口以后随机划分。

其次，标准化参数、GRA关联度和PCA投影矩阵都应该只使用训练集拟合，再应用到验证集和测试集。

如果先使用全部数据计算标准化或PCA参数，就会把测试阶段的信息提前泄漏给模型，导致结果偏高。

11. .pth、ONNX和OM分别是什么？

.pth 是PyTorch模型文件，通常保存网络权重或Checkpoint，依赖PyTorch运行环境。

ONNX是通用模型中间表示，包含计算图、权重、算子和输入输出信息，用于连接训练框架与硬件编译器。

OM是经过ATC针对具体Ascend芯片编译后的离线模型，由ACL Runtime在昇腾NPU上加载和执行。

12. 为什么ONNX导出使用固定Shape？

固定Shape以后，ATC可以提前确定输入输出Buffer、中间张量内存、算子实现和部分图优化策略。

项目的特征维度和窗口长度本身就是固定的，因此将输入固定为 [1, 21, 4] 比较合理。

代价是运行时不能任意改变窗口长度。如果需要支持多种窗口，可以分别编译多个OM，或者使用平台支持的动态档位。

13. 为什么选择opset 12？

opset表示ONNX算子语义集合的版本。

版本太新时，当前CANN工具链可能不支持其中的部分算子；版本太旧又可能无法完整表达当前LSTM结构。

opset 12是当时PyTorch导出和ATC编译之间兼容性相对稳定的选择。它不是越高越好，而是需要与实际部署工具链匹配。

14. 为什么要把 h0/c0 设置成模型输入？

LSTM不仅依赖当前输入，还依赖隐藏状态和细胞状态。

在PyTorch中，这些状态可以由Python运行逻辑动态维护；但OM是静态计算图，不会自动保存两次模型调用之间的状态。

因此，我把 h0/c0 设置成显式输入，把 hn/cn 设置成输出。板端完成一次推理以后，将新的状态保存并传给下一次推理。

15. 滑动窗口和跨窗口状态传递会不会重复历史信息？

会存在这个风险。

相邻21步窗口中有20个循环重复。如果同时把上一个窗口的 hn/cn 传给下一个窗口，部分历史信息可能被重复编码。

当前方案是面向静态OM部署的工程实现。更严谨的做法是比较三种方式：每个窗口零状态、重叠窗口传状态，以及不重叠Chunk传状态，然后根据RMSE和EOL误差选择。

面试中应该主动承认这是当前方案的一个局限。

16. ATC编译主要做了什么？

ATC将通用ONNX计算图编译成面向Ascend 310B4的OM模型。

它会检查算子支持情况，并结合目标芯片、输入Shape和精度模式完成算子实现选择、内存规划和部分图优化。

编译时最关键的是输入名称和Shape必须与ONNX一致，`soc_version`必须与目标芯片匹配。

17. 为什么初版使用FP32？

容量预测属于连续值回归，较小的数值偏差也可能改变EOL越阈位置。

所以我先使用FP32建立迁移精度基线，确认PC模型和板端模型的结果一致，再考虑FP16或INT8。

FP32属于保守精度策略，不是性能优化。是否降低精度应该通过板端时延、RMSE和EOL误差共同决定。

18. ACL Lite和ACL是什么关系？

ACL是昇腾平台底层推理接口，负责设备、Context、Stream、模型、内存和执行管理。

ACL Lite是对ACL常用流程的封装，减少Python推理程序中的样板代码。

我的工作是基于ACL Lite搭建板端推理工程，并管理模型执行和资源生命周期，不是重新实现ACL Runtime。

19. 板端推理的完整流程是什么？

主要流程是：

1. 加载标准化参数和电池数据；
 2. 初始化ACL资源；
 3. 加载OM模型；
 4. 构造 `x`、`h0`、`c0` 三个输入；
 5. 执行NPU推理；
 6. 读取容量预测值和新的 `hn/cn`；
 7. 更新LSTM状态；
 8. 对容量进行反标准化；
 9. 计算RMSE、MAE和EOL误差；
 10. 保存CSV、JSON和结果图；
 11. 释放模型与ACL资源。
-

20. 如何验证模型转换前后结果一致？

我会分三级验证。

第一步，对相同输入比较PyTorch与ONNX输出，确认导出过程没有改变模型语义。

第二步，对相同输入比较ONNX与OM板端输出，检查模型编译和精度模式是否引入明显偏差。

第三步，在完整测试集上比较容量曲线、RMSE、MAE和EOL循环位置。

如果结果不一致，再逐层检查特征顺序、标准化参数、PCA矩阵、输入Shape、数据类型、输出节点和容量反标准化。

21. 如果继续做端侧优化，你会怎么做？

我不会直接选择量化或剪枝，而是先通过Profiling拆分：

- 数据读取和前处理时间；
- Host到Device传输时间；
- OM模型执行时间；
- Device到Host传输时间；
- 后处理时间。

如果模型计算是瓶颈，可以评估FP16、INT8、结构化剪枝、知识蒸馏，或者用GRU、TCN替换LSTM。

如果数据搬运和调度占比高，则优先考虑Buffer复用、减少内存复制和降低Host/Device同步次数。

最终选择必须同时看板端时延、RMSE和EOL误差，而不能只看理论计算量。

22. 这个项目最大的不足是什么？

首先，使用的是NASA实验室公开数据，不能完全代表真实车辆和储能系统中的复杂工况。

其次，当前主要完成了离线数据推理，没有接入真实BMS在线数据流。

另外，固定Shape限制了输入灵活性，滑动窗口与状态传递还需要进一步消融验证，也缺少板端时延、功耗和长期运行等产品化指标。

因此，这个项目证明了算法和昇腾部署链路的可行性，但还不是车规量产系统。

三、困难、协作和复盘问题

1. 你在这个项目中遇到的最大困难是什么？

我认为最大的困难，是把PC端具有动态时序状态的LSTM模型，转换成昇腾NPU能够静态编译和连续执行的OM模型。

普通前馈模型每次输入输出相互独立，但LSTM还依赖 h/c 状态。最初如果只导出数据输入，OM每次调用都会从零状态开始，无法明确维护跨次推理状态。

我的解决方法是修改模型接口，把 h_0/c_0 显式设置为输入，把 h_n/c_n 设置为输出，并把序列长度固定为21。板端每次执行后保存新的状态，再传入下一次推理。

这件事让我认识到，模型部署不能只关注格式转换，还必须保证训练模型与部署模型的计算语义一致。

2. 如果板端结果和PC端不一致，你是怎么排查的？

我的排查思路是按照完整数据链路逐层定位，而不是直接判断NPU计算有问题。

首先检查板端输入是否正确，包括特征顺序、标准化参数、PCA结果、Shape和数据类型；然后比较PyTorch和ONNX在相同输入下的输出；再比较ONNX和OM输出；最后检查容量反标准化和指标计算。

对于LSTM还要额外检查 `h0/c0` 初始化、窗口构造和状态更新是否一致。

这类问题中，真正容易出错的通常不是LSTM算子本身，而是训练和部署阶段的前处理、状态管理或输出解析不一致。

3. 你是怎么和论文合作者及老师协作的？

这个项目中，我主要负责算法实现、实验分析和后续昇腾部署，老师和其他合作者会参与研究方向、实验方案以及论文内容的讨论。

协作过程中，我会把问题拆成能够验证的部分，例如特征筛选是否有效、PCA维度如何确定、不同预测跨度下结果怎样，以及实验结论是否能够被数据支持。

在论文整理时，我负责将算法流程、实验结果和主要结论形成完整材料，再根据老师和合作者的意见调整实验解释与论文表达。

后续的昇腾部署主要由我继续完成，并不属于论文原始实验内容。这样可以明确区分共同研究成果和我个人完成的工程部署工作。

4. 算法研究和工程部署是怎样衔接的？

我不是重新设计一套与论文无关的模型，而是把论文中的算法作为部署起点。

论文阶段先验证GRA + PCA + LSTM方法在电池容量和EOL预测上的有效性；工程阶段再把训练好的模型转换为ONNX和OM，并根据昇腾静态编译要求固定Shape、显式暴露 `h/c` 状态，最后通过ACL Lite在板端执行。

算法侧决定模型输入、网络结构和评价指标，部署侧保证这些语义在NPU上保持一致。两部分是连续关系，而不是两个独立项目。

5. 这个项目给你最大的收获是什么？

最大的收获是，我对AI项目的理解从“训练出一个指标较好的模型”扩展到了“让模型在具体硬件上正确运行”。

算法效果只是第一步，真正部署时还需要考虑模型格式、算子兼容、静态Shape、数据类型、状态传递、Host与Device内存，以及运行时资源生命周期。

我也认识到，端侧优化不能只看模型参数量或理论计算量。最终是否有效，必须同时结合硬件实测性能和业务精度指标。

6. 如果重新做一次，你会优先改进什么？

我会优先改进三个方面。

第一，重新建立更加严格的数据划分与验证流程，例如按电池进行留一验证，所有标准化、GRA和PCA参数只在训练集上拟合，进一步验证跨电池泛化能力。

第二，对滑动窗口和 `h/c` 状态传递进行消融实验，明确零状态窗口、重叠窗口传状态和不重叠Chunk传状态之间的差异。

第三，建立板端Profiling基线，测量前处理、数据搬运、模型执行和后处理耗时，再评估FP16、结构化剪枝、知识蒸馏和轻量模型替换。

这样可以把当前的可运行原型进一步发展为可量化、可比较、可持续优化的端侧推理系统。